

Apache Airflow

What is Apache Airflow?

Apache Airflow is an open-source platform used to programmatically author, schedule, and monitor workflows. Originally developed at Airbnb in 2014 and later donated to the Apache Software Foundation, Airflow has become one of the most popular tools for orchestrating complex data pipelines.

Airflow allows users to define workflows as code using Python, making them maintainable, versionable, testable, and collaborative. Workflows in Airflow are represented as Directed Acyclic Graphs (DAGs), where nodes represent tasks and edges define dependencies between them.

Key capabilities of Apache Airflow include:

- Dynamic pipeline generation using Python code
- Rich user interface to visualize pipelines, monitor progress, and troubleshoot issues
- Extensible architecture supporting custom operators, hooks, and sensors
- Integration with major cloud platforms (AWS, GCP, Azure) and data tools
- Scalable execution through various executor backends

Advantages of Airflow

1. Workflows as Code

Airflow allows workflows to be written in Python, making pipelines flexible, maintainable, and easy to manage.

2. Rich Web UI and Monitoring

Airflow provides a web interface to monitor DAGs, track task status, view logs, and manage workflows.

3. Extensibility and Integrations

Airflow supports integration with databases, cloud platforms, and external services through operators and hooks.

4. Scalability

Airflow can scale from a single system to distributed environments using executors like CeleryExecutor and KubernetesExecutor.

5. Active Community Support

Apache Airflow has strong community support, regular updates, and extensive documentation.

6. Scheduling and Automation

Airflow can schedule workflows automatically at specific time intervals such as hourly, daily, or weekly.

7. Error Handling and Retry Mechanism

Airflow supports task retries, failure alerts, and dependency management to improve workflow reliability.

Airflow Architecture

Apache Airflow follows a modular, distributed architecture composed of several core components that work together to schedule and execute workflows.

Webserver

The Airflow Webserver provides the Flask-based UI that allows users to manage DAGs, view logs, monitor task execution, and configure connections and variables. It reads metadata from the database but does not execute tasks itself.

Scheduler

The Scheduler is the brain of Airflow. It continuously parses DAG files, determines which tasks are ready to run based on their schedule and dependencies, and places them in a queue for the executor to pick up. Airflow 2.x introduced a highly-available, multi-scheduler setup for production reliability.

Executor

The Executor is responsible for running tasks. It acts as an intermediary between the Scheduler and the actual worker processes.

Common executors include `SequentialExecutor` (dev/testing), `LocalExecutor` (single-node), `CeleryExecutor` (multi-node via message broker), and `KubernetesExecutor` (container-based).

Workers

Workers are the processes that actually execute the task logic. In a `CeleryExecutor` setup, worker nodes consume tasks from a Redis or RabbitMQ message queue. In a `KubernetesExecutor` setup, each task runs in its own ephemeral pod.

Metadata Database

Airflow uses a relational database (typically PostgreSQL or MySQL) to store all metadata — DAG definitions, task instance states, logs, variables, connections, and XCom values. It is the single source of truth for the entire system.

DAG Files & DAG Processor

DAG Python files are stored in a directory that Airflow monitors. The DAG Processor parses these files and registers new or updated DAGs into the metadata database for the Scheduler to act on.

What is a DAG and What are Executors?

Directed Acyclic Graphs (DAGs)

A DAG (Directed Acyclic Graph) is the fundamental building block of Airflow. It is a collection of tasks organized in a way that reflects their relationships and dependencies. The key properties are:

- Directed – edges have a direction (task A → task B means A must complete before B starts)
- Acyclic – there are no cycles or loops; a task cannot depend on itself directly or transitively
- Graph – tasks (nodes) are connected by dependency edges

A DAG is defined in a Python file and describes what tasks to run, in what order, and on what schedule. It does not describe how tasks run — that is the Executor's responsibility.

Example DAG structure: `extract_data → transform_data → load_data → send_report`

Executors in Airflow

An Executor defines how and where task instances are run. Airflow ships with several built-in executors suited for different deployment scenarios:

Sequential Executor

Runs one task at a time in the same process as the scheduler. Mainly used for development and testing purposes.

Local Executor

Runs tasks as subprocesses on the same machine and supports parallel execution. Suitable for small to medium workloads.

Celery Executor

Distributes tasks across multiple worker nodes using Redis or RabbitMQ. Commonly used in production environments.

Kubernetes Executor

Runs each task in a separate Kubernetes pod, providing better scalability and resource isolation for cloud deployments.

Celery Kubernetes Executor

A hybrid executor that combines CeleryExecutor and KubernetesExecutor for flexible task execution.